

# LABORATORY OBSERVATION

Student Name: K. Hansikha

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 5-3

---

## 1. TITLE

Express.js Student Server

---

## 2. AIM

To develop a small server using Express.js that responds to browser requests through separate URL routes, returns a collection of student records, provides application information, and reports an appropriate response for an unknown route.

---

## 3. OBJECTIVE

The objectives of this experiment are:

- To initialize an Express.js application on a Node.js runtime.
  - To understand how middleware can be registered before route handling.
  - To create GET routes for the home, student, and about pages.
  - To represent student details as JavaScript objects inside an array.
  - To return structured student records as JSON to a browser client.
  - To handle an invalid URL by returning an HTTP 404 response.
- 

## 4. THEORY

Express.js provides routing and request-response features on top of Node.js. A server application can associate a URL path and an HTTP method with a callback function. When a request matches the route, the callback receives the request object and uses the response object to send data back to the browser.

In this implementation, the root route returns an HTML heading, the /students route serializes five student objects using `res.json()`, and the /about route sends a short HTML description. The application also registers `express.json()` middleware and a final handler that sends status 404 when no earlier route matches.

---

## 5. ALGORITHM / PROCEDURE

1. Import the Express.js package and create the application object.
2. Assign port 3000 to the server.
3. Register `express.json()` middleware.
4. Create a GET handler for the root path.
5. Prepare five student objects with id, name, and course properties.
6. Create a GET /students handler and return the array as JSON.
7. Create a GET /about handler with application information.
8. Add a final middleware function for unknown paths.
9. Start the server using `app.listen()`.
10. Open each URL in a browser and verify the returned response.

## 6. PROGRAM

app.js

```
const students = [
  {
    id: 1,
    name: "Hansi",
    course: "CSM"
  },
  {
    id: 2,
    name: "Anjali",
    course: "CSE"
  },
  {
    id: 3,
    name: "Rahul",
    course: "ECE"
  },
  {
    id: 4,
    name: "Priya",
    course: "IT"
  },
  {
    id: 5,
    name: "Kiran",
    course: "CSM"
  }
];

res.json(students);
});

// About route
app.get("/about", (req, res) => {
  res.send(`
    <h1>About Student Server</h1>
    <p>This is an Express.js application.</p>
    <p>It demonstrates routing and HTTP GET requests.</p>
  `);
});
```

## 6. PROGRAM (CONTINUED)

app.js (continued)

```
});

// Handle invalid routes
app.use((req, res) => {
  res.status(404).send("<h1>404 - Page Not Found</h1>");
});

// Start server
app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

## 7. CODE EXPLANATION

| Code / Method             | Purpose                                                                 |
|---------------------------|-------------------------------------------------------------------------|
| require("express")        | Loads the Express.js framework into the Node.js program.                |
| express()                 | Creates the Express application used to register middleware and routes. |
| app.use(express.json())   | Adds JSON parsing middleware before route processing.                   |
| app.get("/", ...)         | Handles GET requests made to the server root.                           |
| students array            | Stores five student objects that can be returned to the client.         |
| res.json(students)        | Converts the JavaScript array to a JSON response.                       |
| res.send(...)             | Sends an HTML/text response for the home and about routes.              |
| res.status(404).send(...) | Returns a 404 status and message for an unknown path.                   |
| app.listen(...)           | Starts the server and listens on port 3000.                             |

## 8. TEST CASES AND EXECUTION DATA

| Test Case | Route / Action | Expected Result                            | Actual Result                           | Status |
|-----------|----------------|--------------------------------------------|-----------------------------------------|--------|
| TC-01     | Server startup | Server listens on port 3000.               | Server running at http://localhost:3000 | Pass   |
| TC-02     | /              | Home heading is returned.                  | Welcome to Student Server               | Pass   |
| TC-03     | /students      | Five student objects are returned as JSON. | Five records displayed in browser       | Pass   |
| TC-04     | /about         | Application information is displayed.      | About Student Server page displayed     | Pass   |
| TC-05     | /abc           | Unknown route returns HTTP 404.            | 404 - Page Not Found displayed          | Pass   |

---

## 9. OUTPUT

### Output 1: Express.js Server Running in Terminal

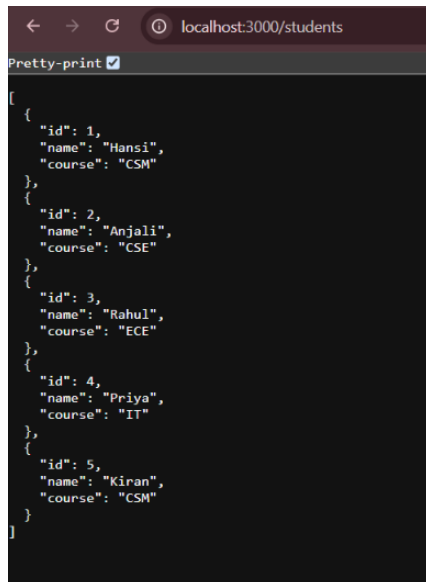
```
PS C:\Users\hansi\OneDrive\Desktop\git-programs\full-stack\PBL\Q3_Express_Student_Server> node app.js
Server running at http://localhost:3000
█
```

### Output 2: Home Route (/)



# Welcome to Student Server

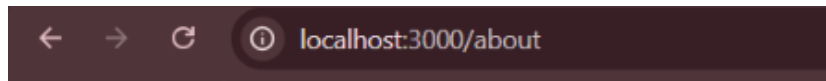
### Output 3: Students Route (/students)



---

## 9. OUTPUT (CONTINUED)

Output 4: About Route (/about)

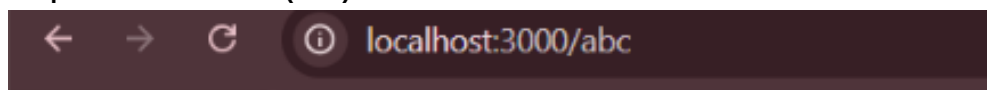


# About Student Server

This is an Express.js application.

It demonstrates routing and HTTP GET requests.

Output 5: Invalid Route (/abc)



# 404 - Page Not Found

---

## 10. RESULT

Thus, the Express.js Student Server was implemented successfully using Node.js and Express.js. The demonstrated execution confirms that the server starts on port 3000, the home route returns its welcome message, the /students route returns five student records in JSON format, the /about route provides application information, and an invalid route receives a 404 response.

**How this would be submitted**

**Source Code**

Q3\_ExpressJS\_Student\_Server/

■■■■ app.js

■■■■ package.json

■■■■ package-lock.json